# AIDRIN Security Review

**Date:** 2026-10-22 **Scope:** Full repository, branch `develop` (commit 779d081) **Method:** Repository-wide audit followed by independent per-finding verification; every finding below was confirmed at confidence ≥ 8/10 by tracing the exact code path. Line numbers refer to the working tree at the commit above.

## Summary

| # | Severity | Category | Location | Finding |
|---|---|---|---|---|
| 1 | High | Insecure deserialization (RCE) | `aidrin/file_handling/readers/npz_reader.py:9` | `.npz` upload unpickles attacker data |
| 2 | High | Hardcoded credential | `web/__init__.py:53` | Default Flask secret key enables session forgery |
| 3 | Medium | Code execution / missing authz | `web/routes/custom.py:210–225, 70–73` | Un-gated custom-metric upload + execute |
| 4 | Medium | Stored XSS | `web/templates/logs.html:118–133` | Log viewer renders unescaped log content |
| 5 | Medium | Information disclosure | `web/routes/admin.py:31–57` | Global server log readable without auth |
| 6 | Medium | Path confinement gap | `web/routes/core.py:287–291` | `send_file` on unconfined session path |

Suggested fix order: 1 → 2 → 6 → 3 and 5 → 4. Findings 1, 2, and 6 together are roughly a ten-line diff.

---

## 1. Pickle deserialization RCE via `.npz` upload (High)

**Location:** `aidrin/file_handling/readers/npz_reader.py:9` **Confidence:** 9/10

```
npz_data = np.load(self.file_path, allow_pickle=True)
```

`allow_pickle=True` unpickles object arrays inside any `.npz` the app reads. `.npz` is an accepted anonymous upload type ( `READER_MAP` and `SUPPORTED_FILE_TYPES` in `aidrin/file_handling/file_parser.py:25,39` ), and every web metric route, the CLI, and the MCP server funnel through this reader. No hop in the pipeline inspects file content; `secure_filename` and `confine_to_upload_folder` sanitize only the path.

**Exploit scenario.** Two unauthenticated requests: (1) POST a crafted `.npz` — a zip containing an `.npy` member whose header declares an object dtype and whose body is a malicious pickle — to `/inspector` with `fileTypeSelector=.npz` ; (2) GET `/summary-statistics` or any metric route. `read_file` dispatches to `npzReader.read()` and the pickle executes arbitrary Python as the Flask/Celery process user. Plain dataset reading is not one of AIDRIN's documented code-execution surfaces, so this exceeds the project's stated trust model.

**Fix.** Change line 9 to `np.load(self.file_path, allow_pickle=False)` (NumPy's default). Legitimate numeric archives still load; pickled archives raise `ValueError`, which the existing `read_file` error handling already converts into the "Unable to read the uploaded file." response.

---

## 2. Hardcoded default Flask secret key (High)

**Location:** `web/__init__.py:53` **Confidence:** 8/10

```
app.secret_key = os.environ.get("AIDRIN_SECRET_KEY", "aidrin")
```

The fallback is a public constant committed to the repository (CWE-1392, insecure default credential). Any deployment that does not explicitly set `AIDRIN_SECRET_KEY` signs its client-side Flask session cookies with `"aidrin"`, so anyone can mint arbitrary session contents. The session carries the app's entire security state: `user_id` (the sole per-user isolation token, `web/routes/utils.py:204–205`), `uploaded_file_path`, `custom_loader_spec`, `globus_tokens` / `globus_authenticated` (`web/routes/globus.py:241,248`), and `llm_config` (`web/routes/llm.py:64`).

**Exploit scenario.** An attacker forges a cookie with another user's `user_id` to access their session data, or sets `custom_loader_spec`, which `build_file_info` (`web/routes/utils.py:103–113`) consumes without re-checking the `ENABLE_WEB_CUSTOM_LOADERS` opt-in, to have the importlib loader path execute an arbitrary `.py` on disk.

**Fix.** Remove the fallback and fail closed:

```
app.secret_key = os.environ.get("AIDRIN_SECRET_KEY")
if not app.secret_key:
    raise RuntimeError("AIDRIN_SECRET_KEY must be set")
```

If zero-config dev startup must keep working, fall back to `secrets.token_hex(32)` (a per-process random key) — never a constant.

---

## 3. Unauthenticated remote code execution via web custom metrics (Medium)

**Location:** `web/routes/custom.py:210–225` (save) and `web/routes/custom.py:70–73` (execute) **Confidence:** 8/10

`/save-custom-metric-text` writes attacker-supplied Python source to disk (`f.write(code)`) and `/custom-metrics` imports and runs it in-process (`spec.loader.exec_module(module)`), with no authentication and no feature flag. The sibling custom-loader web surface, which carries the identical in-process code-execution risk, is deliberately gated behind the opt-in `ENABLE_WEB_CUSTOM_LOADERS` flag with a 403 (`web/__init__.py:181–185`, `web/routes/core.py:57–60`) and a code comment restricting it to trusted deployments. The custom-metric path contradicts the project's own security posture. AGENTS.md documents custom metrics as a trusted-input surface, which is why this is rated Medium rather than High, but the un-gated web exposure is weaker than the project's own loader precedent.

**Exploit scenario.** Four anonymous HTTP requests: GET `/load-custom-metric` for a session, POST malicious `CustomDR` code to `/save-custom-metric-text`, POST any dataset to `/inspector`, POST `/custom-metrics`. Arbitrary code then runs in the Flask process.

**Fix.** Add an `ENABLE_WEB_CUSTOM_METRICS` flag (default off) mirroring the loader pattern in `web/__init__.py:183-185`, and `abort(403)` at the top of the `/save-custom-metric-text` handler and the `/custom-metrics` POST branch when it is unset (about six lines).

---

## 4. Stored XSS in the log viewer (Medium)

**Location:** `web/templates/logs.html:118-133` **Confidence:** 9/10

The `/logs` page builds table rows with `tbody.innerHTML`, interpolating `row.timestamp`, `row.logger`, `row.level`, and `row.message` with no escaping, while the rest of the frontend consistently uses `escapeHtml` / DOMPurify. The log file is process-global across all sessions, and attacker-controlled strings reach it unconditionally: the `fileTypeSelector` form field is stored unvalidated (`web/routes/core.py:140`) and logged verbatim as `Unsupported file type: ...` (`aidrin/file_handling/file_parser.py:334`); filter keys from POST `/filter-file` are logged at `file_parser.py:216`.

**Exploit scenario.** An attacker uploads with `fileTypeSelector=<img src=x onerror=...>`. The markup lands in the shared log and executes in the browser of any user or operator who opens `/logs`, enabling theft of their Flask session cookie or arbitrary actions in the operator's browser. This is stored XSS delivered through finding 5's endpoint, not mere log spoofing.

**Fix.** Escape all four interpolated fields before building row HTML in `logs.html` (a small `esc()` replace helper, or build cells via `document.createElement` / `textContent`). Optionally also whitelist `fileTypeSelector` against `READER_MAP` keys at `core.py:140`, but escaping at the sink is the necessary and sufficient fix since log content is inherently unstructured.

---

## 5. Unauthenticated global log disclosure (Medium)

**Location:** `web/routes/admin.py:31-33` (`/logs`) and `:36-57` (`/view-logs`) **Confidence:** 8/10

Both routes return the entire process-wide `data/logs/aidrin.log` to any anonymous client. There is no decorator, flag, or `before_request` hook anywhere in the app (the only auth in the codebase is Globus OAuth, which gates only the Globus transfer routes). A single root `FileHandler` (`aidrin/logging.py:14-18`) means the log records every user's activity: upload events, server-side file paths including original filenames (`web/routes/core.py:170-174`), dataset parse-error tracebacks, custom-metric tracebacks, and LLM model + `api_base` configuration (`web/routes/llm.py:71`; the API key itself is not logged).

**Exploit scenario.** Anonymous GET `/view-logs` on a shared deployment reveals other users' filenames, paths, errors, and activity, breaking the app's per-session isolation model. It is also the delivery channel for finding 4.

**Fix.** Gate both routes behind an opt-in `AIDRIN_ENABLE_LOG_VIEWER` env flag (mirroring the `ENABLE_WEB_CUSTOM_LOADERS` pattern), aborting 404/403 when unset. Real admin authentication is the fuller fix.

## 6. Missing path confinement in `/retrieve-uploaded-file` (Medium)

**Location:** `web/routes/core.py:287-291` **Confidence:** 9/10 (exploitable when chained with finding 2)

```
uploaded_file_path = session.get("uploaded_file_path")
...
return send_file(uploaded_file_path, as_attachment=True)
```

The route passes the session value straight to `send_file()` after only an `os.path.exists` check. Every other consumer of `session["uploaded_file_path"]` runs it through `confine_to_upload_folder()` (`web/routes/utils.py:36`) — `filter_file` (`core.py:367`), `summary_statistics` (`core.py:542`), `extract_features` (`core.py:676`), and `build_file_info` (`utils.py:84`). That helper's own docstring names forged session cookies as the threat it defends against; this route is the single deviation, and the one that returns raw file bytes. Relatedly, `session["custom_loader_spec"]` is executed at consumption time without re-checking `ENABLE_WEB_CUSTOM_LOADERS` or confining the loader path to `CUSTOM_LOADERS_FOLDER`.

**Exploit scenario.** With a forged cookie (finding 2), set `uploaded_file_path=/etc/passwd` or the app's `.env` and download it; or point `custom_loader_spec` at any `.py` on disk for code execution.

**Fix.** One line in the retrieve route: `uploaded_file_path = confine_to_upload_folder(session.get("uploaded_file_path"))` (an empty result falls through to the existing 404 branches). Additionally, ignore a session-sourced `custom_loader_spec` unless the flag is on and the spec's path realpath-resolves inside `CUSTOM_LOADERS_FOLDER`.

---

## Checked and cleared

No finding in: `confine_to_upload_folder` itself (realpath + commonpath containment, `web/routes/utils.py:36-61`); upload filename handling (`secure_filename` + UUID, `web/routes/core.py:69-78`); the `send_from_directory` routes (`serve_image`, `serve_docs`, `download_remedy`, `sample_data` — traversal-safe, `sample_data` allowlists type); the Globus submit flow (session-gated, remote paths only); the flag-gated web custom loaders and the agentic `exec()` (`aidrin/agentic/executor.py:123`) — both documented trusted-input surfaces; the metric-result `innerHTML` in `main.js` (self-XSS of own-session data only).